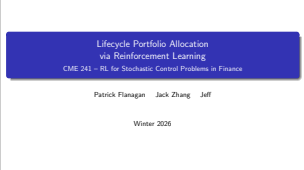
Lifecycle Portfolio Allocation via Reinforcement Learning

CME 241 – RL for Stochastic Control Problems in Finance

Patrick Flanagan Jack Zhang Jeff

Winter 2026

Lifecycle Portfolio Allocationvia Reinforcement Learning



[Patrick speaks – 15 sec]

"Hi everyone, we're Patrick, Jack, and Jeff. Our project applies reinforcement learning to the lifecycle portfolio allocation problem – basically, how should someone invest and consume over their working life to maximize lifetime happiness. We'll walk through the problem setup, our exact DP solution, why it breaks down, and how four different RL algorithms handle the harder version."

Outline

1. Problem: Lifecycle Portfolio Allocation as an MDP
2. Phase 2: Exact DP Solution (Backward Induction)
3. The Curse of Dimensionality
4. Phase 3: Reinforcement Learning Approach
5. Exploration–Exploitation and the ϵ-Greedy Schedule
6. Four RL Agents: Linear Q, SARSA, DQN, PPO
7. Deep Dive: SARSA (On-Policy TD Control)
8. Results and Comparison
9. Conclusions

Outline

- Problem: Lifecycle Portfolio Allocation as an MDP
- Phase 2: Exact DP Solution (Backward Induction)
- The Curse of Dimensionality
- Phase 3: Reinforcement Learning Approach
- Exploration–Exploitation and the ϵ–Greedy Schedule
- Four RL Agents: Linear Q, SARSA, DQN, PPO
- Deep Dive: SARSA (On-Policy TD Control)
- Results and Comparison
- Conclusions

[Jack speaks – 10 sec]

"Here's our roadmap. We'll start with the problem formulation, show the exact solution and its limitations, then spend most of our time on the RL methods and results."

The Lifecycle Allocation Problem

Setting: An investor with T years until retirement must decide each year:

- How much wealth to **consume** (spend) vs. save
- How to **allocate** savings across stocks, bonds, and cash

Stochastic elements:

- **Income** follows a Markov chain (can be low, medium, or high)
- **Market regime** follows a Markov chain (bear, normal, or bull)
- **Asset returns** are random, conditioned on the current regime

Why this matters:

- This is exactly the problem that target-date retirement funds try to solve
- Traditional approaches use fixed rules (e.g., "60/40 stocks/bonds")
- Can RL discover better, *state-dependent* strategies?

The Lifecycle Allocation Problem

Setting: An investor with T years until retirement must decide each year:

- How much wealth to **consume** (spend) vs. save
- How to **allocate** savings across stocks, bonds, and cash

Stochastic elements:

- **Income** follows a Markov chain (can be low, medium, or high)
- **Market regime** follows a Markov chain (bear, normal, or bull)
- **Asset returns** are random, conditioned on the current regime

Why this matters:

- This is exactly the problem that target-date retirement funds try to solve
- Traditional approaches use fixed rules (e.g., "60/40 stocks/bonds")
- Can RL discover better, *state-dependent* strategies?

[Jack speaks – 1 min]

"So what's the problem? Imagine you're 30 and want to retire at 65 or 70. Every year you get income, you decide how much to spend versus save, and you choose how to invest your savings. The tricky part is that everything is random – your income can change, the market can crash or boom, and your investment returns depend on these conditions."

"This is literally the problem that every target-date fund at Vanguard or Fidelity is trying to solve. They typically use simple rules like start with 80% stocks and gradually shift to bonds. We're asking: can reinforcement learning find something better?"

MDP Formulation

State: $s_t = (t,\ W_t,\ y_t,\ z_t)$

- t = time period (year)
- W_t = current wealth
- $y_t \in \{\text{low, med, high}\}$ = income state
- $z_t \in \{\text{bear, normal, bull}\}$ = market regime

Action: $a_t = (\alpha_t,\ \mathbf{w}_t)$

- α_t = consumption fraction (0%–20%)
- $\mathbf{w}_t$ = portfolio weights summing to 1

Reward: $r_t = \beta^t \cdot u(c_t)$

Objective:

$$\max_{\pi}\ \mathbb{E}\left[\sum_{t=0}^{T-1} \beta^t u(c_t) + \beta^T u(W_T)\right]$$

CRRA Utility ($\gamma = 2$):

$$u(c) = \frac{c^{1-\gamma}}{1-\gamma} = -\frac{1}{c}$$

Key property: Diminishing returns. Going from \$10→\$20 matters much more than \$100→\$110.

$\Rightarrow$ Agent is **risk-averse**

MDP Formulation

State: $s_t = (t,\ W_t,\ y_t,\ z_t)$

- t = time period (year)
- W_t = current wealth
- $y_t \in$ {low, mid, high} = income state
- $z_t \in$ {bear, normal, bull} = market regime

Action: $a_t = (\alpha_t,\ \mathbf{w}_t)$

- α_t = consumption fraction (0%–20%)
- $\mathbf{w}_t$ = portfolio weights summing to 1

Reward: $r_t = \beta^t \cdot u(c_t)$

Objective:

$$\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{T-1} \beta^t u(c_t) + \beta^T u(W_T)\right]$$

CRRA Utility ($\gamma = 2$):

$$u(c) = \frac{c^{1-\gamma}}{1-\gamma} = -\frac{1}{c}$$

Key property: Diminishing returns. Going from \$10→\$20 matters much more than \$100→\$110.

⇒ Agent is **risk-averse**

[Jack speaks – 1.5 min]

"We formulate this as a Markov Decision Process. The state captures everything relevant: what year it is, how much wealth you have, your income level, and the market regime. The action is a pair: what fraction to consume, and how to split the rest across stocks, bonds, and cash."

"The reward is CRRA utility of consumption, discounted by beta to the t. With gamma equals 2, the utility function is negative 1 over c. This means the agent is risk-averse – losing 50 dollars hurts a lot more than gaining 50 dollars feels good. The marginal value of an extra dollar decreases as you get richer. This is important because it means the optimal strategy is NOT just to maximize wealth – it's to balance growth against safety."

"At the terminal period, we also get utility from whatever wealth remains – think of this as the value of your retirement nest egg."

Transition Dynamics

Wealth transition:

$$W_{t+1} = \underbrace{(W_t - c_t)}_{\text{savings}} \cdot \underbrace{(\mathbf{w}_t \cdot \mathbf{R}_t)}_{\text{portfolio return}} + \underbrace{\text{income}(y_{t+1})}_{\text{labor income}}$$

Income transitions (Markov chain):

$$P_y = \begin{pmatrix} 0.70 & 0.25 & 0.05 \\ 0.15 & 0.70 & 0.15 \\ 0.05 & 0.25 & 0.70 \end{pmatrix} \quad \text{Income values: \$3, \$10, \$25}$$

Regime transitions (Markov chain):

$$P_z = \begin{pmatrix} 0.50 & 0.40 & 0.10 \\ 0.20 & 0.60 & 0.20 \\ 0.10 & 0.40 & 0.50 \end{pmatrix} \quad \text{Regimes: Bear, Normal, Bull}$$

Asset returns: 3 scenarios per regime (bad/medium/good) with regime-dependent probabilities. E.g., in a bear market, stocks return 0.82–1.02×; in bull, 0.98–1.25×.

Transition Dynamics

Wealth transition:

$$W_{t+1} = \underbrace{(W_t - c_t)}_{\text{savings}} \cdot \underbrace{(\mathbf{w}_t \cdot \mathbf{R}_t)}_{\text{portfolio return}} + \underbrace{\text{income}(y_{t+1})}_{\text{labor income}}$$

Income transitions (Markov chain):

$$P_y = \begin{pmatrix} 0.70 & 0.25 & 0.05 \\ 0.15 & 0.70 & 0.15 \\ 0.05 & 0.25 & 0.70 \end{pmatrix} \quad \text{Income values: \$3, \$10, \$25}$$

Regime transitions (Markov chain):

$$P_z = \begin{pmatrix} 0.50 & 0.40 & 0.10 \\ 0.20 & 0.60 & 0.20 \\ 0.10 & 0.40 & 0.50 \end{pmatrix} \quad \text{Regimes: Bear, Normal, Bull}$$

Asset returns: 3 scenarios per regime (bad/medium/good) with regime-dependent probabilities. E.g., in a bear market, stocks return 0.82–1.02×; in bull, 0.98–1.25×.

[Jeff speaks – 1 min]

"Here's how the world evolves. Your wealth next period equals your savings times your portfolio return, plus whatever income you earn. Both income and the market regime follow Markov chains – they're persistent but random. If you're in a bear market, there's a 50% chance you stay in a bear market next year."

"Asset returns depend on the regime. In a bear market, stocks can lose 18% in a bad scenario. In a bull market, stocks can gain 25%. Bonds and cash are much more stable. This regime-dependence is what makes the problem interesting – a smart agent should invest differently in bull versus bear markets."

Phase 2: Exact Solution via Backward Induction

Bellman equation:

$$V_t(w, y, z) = \max_{(\alpha, \mathbf{w})} \left[u(\alpha w) + \beta \sum_{y', z', k} p_{y'|y}\, p_{z'|z}\, p_k^{(z)}\; V_{t+1}(w', y', z') \right]$$

Algorithm:

1. Set $V_T(w) = u(w)$ for all terminal states
2. For $t = T-1, T-2, \ldots, 0$:
   - For every state (w_i, y, z): try all 165 actions, pick the best
   - Store optimal value and policy

Phase 2 problem size:

- 121 wealth $\times$ 2 income $\times$ 2 regime = 484 states/period
- $\times$ 25 periods $\times$ 165 actions
- = **2.0 million** DP cells
- Solved in **0.4 seconds**

Key findings:

- DP-optimal beats all baselines on utility
- High stock allocation when young
- Shifts to bonds near retirement
- Consumption increases over time

Phase 2: Exact Solution via Backward Induction

Bellman equation:

$$V_t(w, y, z) = \max_{(\alpha,\omega)} \left[u(\alpha w) + \beta \sum_{y',z',k} p_{y'|y}\, p_{z'|z}\, p_k^{(t)}\, V_{t+1}(w', y', z') \right]$$

Algorithm:

1. Set $V_T(w) = u(w)$ for all terminal states
2. For $t = T-1, T-2, \ldots, 0$:
   - For every state (w, y, z): try all 165 actions, pick the best
   - Store optimal value and policy

Phase 2 problem size:

- 121 wealth $\times$ 2 income $\times$ 2 regime $=$ 484 states/period
- $\times$ 25 periods $\times$ 165 actions
- $\approx$ **2.0 million** DP cells
- Solved in **0.4 seconds**

Key findings:

- DP-optimal beats all baselines on utility
- High stock allocation when young
- Shifts to bonds near retirement
- Consumption increases over time

[Jack speaks – 1.5 min]

"Phase 2 gives us the exact solution using backward induction – the standard dynamic programming approach from Chapter 4 of the textbook. We start at retirement, where value is just utility of your remaining wealth. Then we work backward: at each year, for every possible state, we try all 165 actions and pick the one that maximizes immediate utility plus discounted expected future value."

"With 484 states per period and 25 periods, this is about 2 million calculations total – done in under half a second. The optimal policy makes intuitive sense: invest heavily in stocks when you're young because you have time to recover from crashes, and shift toward bonds as you approach retirement. It beats every static baseline we tested."

"But this only works because we kept the problem small. What happens when we make it more realistic?"

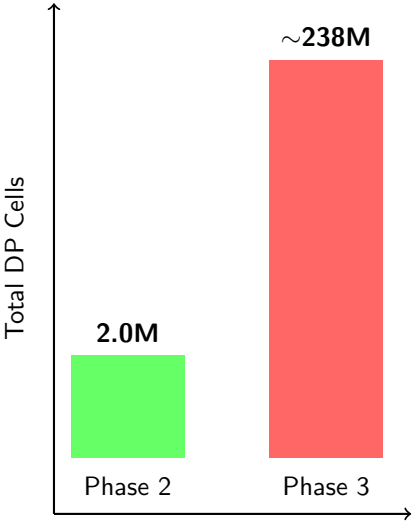
The Curse of Dimensionality

Making the problem *slightly* more realistic:

	Phase 2	**Phase 3**
Horizon	25	40
Wealth	121 grid pts	continuous
Income states	2	3
Regime states	2	3
Alloc. increments	25%	20%
Actions/state	165	210
DP cells (if discretized)†	**2.0M**	**∼238M**

† Hypothetical: 500-pt wealth grid, 10% alloc. increments for Phase 3.

119× blowup – and this is still just 3 assets!



Solution: Function approximation.

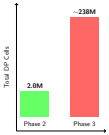
The Curse of Dimensionality

Making the problem *slightly* more realistic:

	Phase 2	Phase 3
Horizon	25	40
Wealth	121 grid pts	continuous
Income states	2	3
Regime states	2	3
Alloc. increments	25%	20%
Actions/state	165	210
DP cells (if discretized)[1]	**2.0M**	**~238M**

[1] Hypothetical: 500-pt wealth grid, 10% alloc. increments for Phase 3.

119× **blowup** – and this is still just 3 assets!



Solution: Function approximation

[Jeff speaks – 1 min]

"Here's why we need RL. We just added one more income state, one more regime, made the wealth continuous, and extended the horizon from 25 to 40 years. The DP table blows up by 119x to nearly 240 million cells. And this is still only 3 assets! If we wanted 5 or 10 asset classes, it would be completely infeasible."

"The solution is function approximation. Instead of storing a value for every single state, we learn a function Q of s, a parameterized by weights w. The agent learns this function by interacting with the environment – trial and error over thousands of simulated lifetimes. This is reinforcement learning."

Phase 3: RL Architecture

Environment (`PortfolioEnv`) – Gym-like interface:

- `reset()` → initial state (year 0, \$100 wealth, medium income, normal market)
- `step(action)` → (next_state, reward, done, info)
- State featurized as 8-dim vector: $\left[\underbrace{t/T}_{\text{time}}, \underbrace{\log W/\log W_{\max}}_{\text{wealth}}, \underbrace{\text{onehot}(y)}_{\text{3 bits}}, \underbrace{\text{onehot}(z)}_{\text{3 bits}}\right]$

Four RL agents:

	Linear Q	**SARSA**	**DQN**	**PPO**
Method	Q-learning	SARSA TD	Dueling DQN	Actor-Critic
Approx.	Linear (24-dim)	Linear (24-dim)	NN (256-256)	NN (256-256)
On/Off-policy	Off	**On**	Off	On
Parameters	5,040	5,040	∼122K	∼191K
Training	15K ep, 18s	15K ep, 15s	15K ep, 23 min	15K ep, 2 min

Linear Q, SARSA, DQN: $\epsilon = 1.0 \to 0.05$ over 8K episodes. PPO: $\epsilon = 0.3 \to 0.0$ over 5K episodes (primarily entropy-regularized).

Phase 3: RL Architecture

Environment (`PortfolioEnv`) – Gym-like interface:

- `reset()` → initial state (year 0, 1100 wealth, medium income, normal market)
- `step(action)` → (next_state, reward, done, info)
- State featurized as 8-dim vector: $\left[\underbrace{t/T}_{\text{time}},\ \underbrace{\log W/\log W_{\max}}_{\text{wealth}},\ \underbrace{\text{onehot}(y)}_{3\text{ bins}},\ \underbrace{\text{onehot}(x)}_{3\text{ bins}}\right]$

Four RL agents:

	Linear Q	SARSA	DQN	PPO
Method	Q-learning	SARSA TD	Dueling DQN	Actor-Critic
Approx.	Linear (24-dim)	Linear (24-dim)	NN (256-256)	NN (256-256)
On/Off-policy	Off	**On**	Off	On
Parameters	5,040	5,040	~122K	~191K
Training	15K ep, 18s	15K ep, 15s	15K ep, 23 min	15K ep, 2 min

Linear Q, SARSA, DQN: $\epsilon = 1.0 \to 0.05$ over 8K episodes. PPO: $\epsilon = 0.3 \to 0.0$ over 5K episodes (primarily entropy-regularized).

[Patrick speaks – 1.5 min]

"For Phase 3, we built a Gym-like environment where agents can simulate lifetimes. The state gets featurized into 8 numbers: normalized time, log-normalized wealth, and one-hot vectors for income and regime. This is what the agents see."

"We implemented four different RL agents. Linear Q-learning and SARSA both use linear function approximation with 24 features – the same raw 8-dim state expanded with polynomial terms and time-regime, wealth-income cross products. This lets the linear model capture nonlinear interactions. DQN uses a dueling Q-network with 256-unit hidden layers. PPO uses separate actor and critic networks – we tried a shared trunk first but the critic loss dominated and prevented the actor from learning. Separating them fixed this."

"The key distinction is on-policy versus off-policy. Linear Q and DQN are off-policy – they bootstrap from the greedy action regardless of what they actually did. SARSA and PPO are on-policy – they learn about the policy they're actually executing. This difference shows up empirically: Linear Q slightly outperforms SARSA because off-policy updates are more aggressive."

Exploration vs. Exploitation

The core RL dilemma: Should the agent *exploit* its current knowledge (pick the best-known action) or *explore* (try something new to learn more)?

ϵ-Greedy policy:

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|} & \text{if } a = \arg\max_{a'} Q(s, a') \\ \frac{\epsilon}{|\mathcal{A}|} & \text{otherwise} \end{cases}$$

GLIE schedule (Greedy in the Limit with Infinite Exploration):

- Start: $\epsilon = 1.0$ (fully random – explore the entire action space)
- Decay linearly over 8,000 episodes (Linear Q, SARSA, DQN) or 5,000 (PPO)
- End: $\epsilon = 0.05$ (mostly greedy, with 5% exploration)
- **Why GLIE matters:** Guarantees every state-action pair is visited infinitely often (exploration) while the policy converges to greedy (exploitation). This is a *necessary condition* for SARSA convergence (Theorem 12.4.1).

Financial intuition: Early episodes are like a junior analyst trying random strategies; later episodes are a seasoned portfolio manager making targeted adjustments.

Exploration vs. Exploitation

The core RL dilemma: Should the agent *exploit* its current knowledge (pick the best-known action) or *explore* (try something new to learn more)?

ϵ-Greedy policy:

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|} & \text{if } a = \arg\max_{a'} Q(s, a') \\ \frac{\epsilon}{|\mathcal{A}|} & \text{otherwise} \end{cases}$$

GLIE schedule (Greedy in the Limit with Infinite Exploration):

- Start: $\epsilon = 1.0$ (fully random – explore the entire action space)
- Decay linearly over 8,000 episodes (Linear Q, SARSA, DQN) or 5,000 (PPO)
- End: $\epsilon \approx 0.05$ (mostly greedy, with 5% exploration)
- **Why GLIE matters:** Guarantees every state-action pair is visited infinitely often (exploration) while the policy converges to greedy (exploitation). This is a *necessary condition* for SARSA convergence (Theorem 12.4.1).

Financial intuition: Early episodes are like a junior analyst trying random strategies; later episodes are a seasoned portfolio manager making targeted adjustments.

[Patrick speaks – 1 min]

"Before diving into the algorithms, let's talk about exploration versus exploitation – one of the central ideas in RL. The agent faces a dilemma: should it take the action it thinks is best, or try something different to potentially discover something better?"

"We use epsilon-greedy: with probability epsilon, take a random action; otherwise take the greedy action. The key is the schedule. We start at epsilon equals 1, meaning fully random. This ensures the agent sees a wide variety of states and actions early on. We decay linearly to 0.05 over 8,000 episodes for Linear Q, SARSA, and DQN – so by episode 8,000 the agents are mostly exploiting. PPO decays from 0.3 to 0 over 5,000 episodes because it's primarily entropy-regularized."

"This is a GLIE schedule – Greedy in the Limit with Infinite Exploration. It's a necessary condition for the SARSA convergence theorem. The financial analogy: early training is like a junior analyst trying wild strategies to see what works; late training is a seasoned manager making small, informed adjustments."

DQN: Deep Q-Network (Ch. 12.6 + Deep Learning)

Key idea: Replace linear $Q(s, a; \mathbf{w}) = \mathbf{w}_a^\top \phi(s)$ with a neural network that can capture *nonlinear* value functions.

Two innovations that make it work:

1. **Experience Replay Buffer** (500,000 transitions)
   - Store $(s, a, r, s', \text{done})$ tuples as the agent interacts
   - Sample random mini-batches (128) for training
   - Breaks temporal correlation $\rightarrow$ more stable gradient updates
   - Reuses data $\rightarrow$ more sample-efficient
2. **Target Network** (updated every 1,000 steps)
   - Separate copy of the Q-network, $Q(s, a; \mathbf{w}^-)$, used in the TD target
   - Target: $r + \gamma \max_{a'} Q(s', a'; \mathbf{w}^-)$ – uses *frozen* weights
   - Prevents the "moving target" problem where the target shifts with every gradient step

Architecture (Dueling DQN): Shared trunk ($8 \rightarrow 256 \rightarrow 256$, Tanh), then $\rightarrow$ Value stream ($256 \rightarrow 1$) and Advantage stream ($256 \rightarrow 210$). $Q = V + A - \bar{A}$. Total: $\sim$122K parameters.

DQN: Deep Q-Network (Ch. 12.6 + Deep Learning)

Key idea: Replace linear $Q(s, a; \mathbf{w}) = \mathbf{w}_a^\top \phi(s)$ with a neural network that can capture *nonlinear* value functions.

Two innovations that make it work:

1. **Experience Replay Buffer** (500,000 transitions)
   - Store $(s, a, r, s', \text{done})$ tuples as the agent interacts
   - Sample random mini-batches (128) for training
   - Breaks temporal correlation → more stable gradient updates
   - Reuses data → more sample-efficient
2. **Target Network** (updated every 1,000 steps)
   - Separate copy of the Q-network, $Q(s, a; \mathbf{w}^-)$, used in the TD target
   - Target: $r + \gamma \max_{a'} Q(s', a'; \mathbf{w}^-)$ – uses frozen weights
   - Prevents the "moving target" problem where the target shifts with every gradient step

Architecture (Dueling DQN): Shared trunk ($8 \to 256 \to 256$, Tanh), then → Value stream ($256 \to 1$) and Advantage stream ($256 \to 210$). $Q = V + A - \bar{A}$. Total: ~122K parameters.

[Jack speaks – 1.5 min]

"DQN extends Q-learning by replacing the linear function approximator with a neural network. The challenge is that naive neural network Q-learning is unstable – the network chases a moving target and the correlated sequential data causes divergence."

"DQN solves this with two innovations. First, an experience replay buffer: instead of learning from transitions in order, we store the last 500,000 transitions and sample random mini-batches of 128. This breaks the temporal correlation between samples and reuses transitions efficiently."

"Second, a target network: a frozen copy of the Q-network that's only updated every 1,000 steps. The TD target uses this frozen network, so the target doesn't shift with every gradient update. Together, these make training stable."

"We use a dueling architecture. The shared trunk takes 8 inputs through two 256-unit Tanh layers. Then it splits into two heads: a value stream that estimates how good state s is overall, and an advantage stream that estimates how much better action a is compared to the average. The final Q-value is V plus A minus the mean advantage – this stabilizes training by separating

PPO: Proximal Policy Optimization (Ch. 13)

Key idea: Instead of learning a value function and deriving the policy, *directly optimize the policy* $\pi_\theta(a|s)$.

Policy gradient theorem (Ch. 13.1):

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) \cdot A^\pi(s,a)]$$

where $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ is the **advantage** (how much better action a is than average).

PPO's clipped surrogate objective (prevents destructively large updates):

$$L(\theta) = \mathbb{E}\left[\min\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}A,\ \ \text{clip}\left(\frac{\pi_\theta}{\pi_{\theta_{\text{old}}}}, 1-\epsilon, 1+\epsilon\right)A\right)\right]$$

Actor-Critic architecture:

- **Actor** $\pi_\theta(a|s)$: outputs action probabilities (policy network)
- **Critic** $V_\phi(s)$: estimates state value (used to compute advantage via GAE)
- **Separate** networks (actor: $\sim$122K params, critic: $\sim$68K params). Total: $\sim$191K.
- Both use 256-256 hidden layers with Tanh and orthogonal initialization.

PPO: Proximal Policy Optimization (Ch. 13)

Key idea: Instead of learning a value function and deriving the policy, *directly optimize the policy* $\pi_\theta(a|s)$.

Policy gradient theorem (Ch. 13.1):

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) \cdot A^\pi(s,a)]$$

where $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ is the **advantage** (how much better action a is than average).

PPO's clipped surrogate objective (prevents destructively large updates):

$$L(\theta) = \mathbb{E}\left[\min\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_{old}}(a|s)} A,\ \mathrm{clip}\left(\frac{\pi_\theta}{\pi_{\theta_{old}}}, 1-\epsilon, 1+\epsilon\right) A\right)\right]$$

Actor-Critic architecture:

- **Actor** $\pi_\theta(a|s)$: outputs action probabilities (*policy network*)
- **Critic** $V_\phi(s)$: estimates state value (used to compute advantage via GAE)
- **Separate** networks (actor: ~122K params, critic: ~68K params). Total: ~191K.
- Both use 256-256 hidden layers with Tanh and orthogonal initialization.

[Jack speaks – 1.5 min]

"PPO takes a completely different approach from the value-based methods. Instead of learning Q-values and deriving the policy, PPO directly parameterizes the policy as a neural network and optimizes it using the policy gradient theorem from Chapter 13."

"The policy gradient says: increase the probability of actions that have positive advantage – meaning they're better than average for that state. The advantage is estimated using Generalized Advantage Estimation, or GAE, which uses the critic's value function."

"The key innovation in PPO is the clipped surrogate objective. Vanilla policy gradient can make huge updates that destroy the policy. PPO clips the probability ratio between the new and old policy to be within 1 plus or minus epsilon – in our case 0.2. This keeps updates conservative and stable."

"One critical implementation detail: we tried a shared-trunk actor-critic first, where both actor and critic share the first two layers. That failed – the critic's value loss dominated the gradient and prevented the actor from learning meaningful action distributions. Splitting into completely

SARSA vs Q-Learning: The Core Difference

SARSA = **S**tate – **A**ction – **R**eward – **S**tate – **A**ction

Both algorithms update $Q(S, A)$ using a TD target. The *only* difference:

Q-Learning (off-policy)

$$\boxed{\Delta \mathbf{w} = \alpha\big(R + \gamma \cdot \max_a Q(S', a) - Q(S, A)\big)\nabla Q}$$

"What's the *best* I *could* do next?"

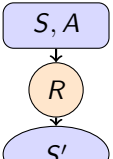
- Assumes future actions are optimal
- Can overestimate Q-values
- More sample-efficient

SARSA (on-policy)

$$\boxed{\Delta \mathbf{w} = \alpha\big(R + \gamma \cdot Q(S', A') - Q(S, A)\big)\nabla Q}$$

"What will I *actually* do next?"

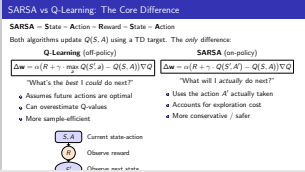
- Uses the action A' actually taken
- Accounts for exploration cost
- More conservative / safer



Current state-action

Observe reward

Observe next state



[Patrick speaks – 2 min]

"This is the heart of my contribution. SARSA and Q-learning have exactly the same setup – same features, same weights, same epsilon-greedy exploration. The only difference is one line in the update equation."

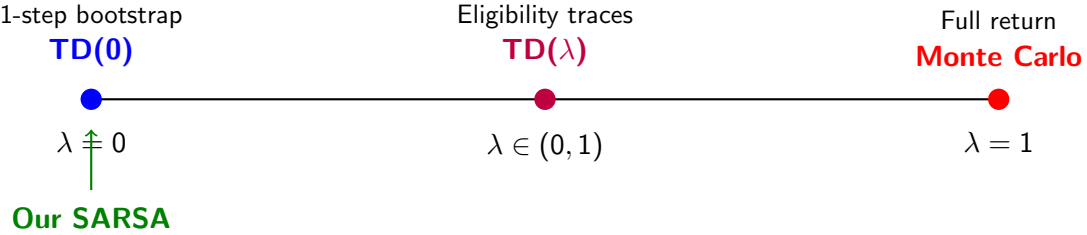
"Q-learning uses max over all actions of Q at the next state. It's asking: what's the best thing I could possibly do next? This is optimistic – it ignores the fact that during training, you'll sometimes take random exploratory actions that are bad."

"SARSA uses Q of s-prime, a-prime, where a-prime is the action you ACTUALLY take next – including random exploration. So it's asking: given that I'll sometimes explore randomly, what's my realistic expected future value?"

"This makes SARSA more conservative. In a financial context, this is arguably better – you don't want your policy to assume you'll always act optimally in the future, because in reality you won't. The name SARSA comes from the five quantities used in each update: State, Action, Reward, State, Action."

Where SARSA Fits: The TD Spectrum

Temporal Difference methods form a spectrum parameterized by $\lambda \in [0, 1]$:



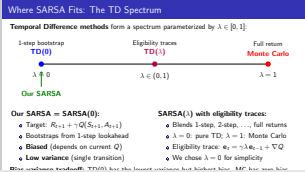
Our SARSA = SARSA(0):

- Target: $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$
- Bootstraps from 1-step lookahead
- **Biased** (depends on current Q)
- **Low variance** (single transition)

SARSA(λ) with eligibility traces:

- Blends 1-step, 2-step, ..., full returns
- $\lambda = 0$: pure TD; $\lambda = 1$: Monte Carlo
- Eligibility trace: $\mathbf{e}_t = \gamma\lambda\,\mathbf{e}_{t-1} + \nabla Q$
- We chose $\lambda = 0$ for simplicity

Bias-variance tradeoff: TD(0) has the lowest variance but highest bias. MC has zero bias



[Patrick speaks – 1 min]

"It's important to understand where our SARSA sits in the bigger picture. TD methods form a spectrum. At one end is TD(0) – our SARSA – which bootstraps from a single step. At the other end is Monte Carlo, which waits until the end of the episode to compute the full return."

"In between is TD-lambda, which uses eligibility traces to blend different-length returns. When lambda is 0, you get pure TD; when lambda is 1, you recover Monte Carlo. We chose lambda equals 0 because our episodes are 40 steps long, and with that horizon, the variance of Monte Carlo returns is very high. TD(0) trades some bias for much lower variance, which means faster and more stable learning in practice."

"If we wanted to improve further, we could try intermediate lambda values – but for this project, SARSA(0) already performs well."

SARSA: Theoretical Properties

Theorem 12.4.1 (Convergence of Tabular SARSA):

SARSA converges to the optimal action-value function $Q(s, a) \to Q^*(s, a)$ under:

1. **GLIE** schedule: $\epsilon_k \to 0$, but every (s, a) visited infinitely often
2. **Robbins-Monro** step sizes:

$$\sum_{t=1}^{\infty} \alpha_t = \infty \qquad \text{and} \qquad \sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

With function approximation (our case):

- No convergence guarantee in general (the "deadly triad": function approximation + bootstrapping + off-policy / on-policy)
- Linear FA with on-policy updates (SARSA) is the **safest** combination – avoids the instability that plagues off-policy + nonlinear FA
- We use a small constant $\alpha = 10^{-5}$ with TD-error clipping at ± 10 – violates Robbins-Monro but is standard practice
- **Discount convention:** $\gamma = 1$ in TD update because β^t is already baked into the reward $r_t = \beta^t u(c_t)$

SARSA: Theoretical Properties

Theorem 12.4.1 (Convergence of Tabular SARSA):

SARSA converges to the optimal action-value function $Q(s,a) \to Q^*(s,a)$ under:

1. **GLIE** schedule: $\epsilon_k \to 0$, but every (s,a) visited infinitely often
2. **Robbins-Monro** step sizes:

$$\sum_{k=1}^{\infty} \alpha_t = \infty \quad \text{and} \quad \sum_{k=1}^{\infty} \alpha_t^2 < \infty$$

With function approximation (our case):

- No convergence guarantee in general (the "deadly triad": function approximation + bootstrapping + off-policy / on-policy)
- Linear FA with on-policy updates (SARSA) is the **safest** combination – avoids the instability that plagues off-policy + nonlinear FA
- We use a small constant $\alpha = 10^{-5}$ with TD-error clipping at ± 10 – violates Robbins-Monro but is standard practice
- **Discount convention:** $\gamma = 1$ in TD update because β^t is already baked into the reward $r_t = \beta^t u(c_t)$

[Patrick speaks – 1 min]

"The textbook gives us a convergence theorem for tabular SARSA. Under two conditions – GLIE exploration and Robbins-Monro step sizes – SARSA converges to the true optimal Q function."

"With function approximation, things are trickier. The deadly triad – function approximation plus bootstrapping plus off-policy learning – can cause divergence. SARSA with linear function approximation is the safest combination because it's on-policy and linear. This avoids the instability that DQN has to work around with its replay buffer and target network."

"One subtlety on discounting: we use gamma equals 1 in the TD update, not 0.96. That's because beta to the t is already multiplied into each reward. So the reward at time t is already discounted by 0.96 to the t. If we also used gamma equals 0.96 in the update, we'd be double-discounting. The TD update with gamma equals 1 correctly implements the discounted CRRA objective."

"We use a constant learning rate alpha equals 10 to the minus 5. This technically violates Robbins-Monro, but is standard practice. The TD-error clipping at plus or minus 10 provides

SARSA: Implementation

Function approximation: $Q(s, a; \mathbf{w}) = \mathbf{w}_a^\top \phi(s)$

Feature vector with polynomial and interaction terms:

$$\phi(s) = [\underbrace{s_{1:8}}_{\text{8 raw}}, \underbrace{t^2, w^2, tw}_{\text{3 poly}}, \underbrace{t \cdot \text{onehot}(y),\ t \cdot \text{onehot}(z)}_{\text{6 time-regime crosses}}, \underbrace{w \cdot \text{onehot}(y),\ w \cdot \text{onehot}(z)}_{\text{6 wealth-regime crosses}}, \underbrace{1}_{\text{bias}}] \in$$

Separate $\mathbf{w}_a \in \mathbb{R}^{24}$ for each of 210 actions. Total parameters: $210 \times 24 = 5{,}040$.

The SARSA invariant – A' in update must equal A' executed:

```
def _expand_features(self, state):
    t, w = state[0], state[1]; cat = state[2:]   # 6 one-hots
    poly = [t**2, w**2, t*w]
    return np.concatenate([state, poly, t*cat, w*cat, [1.0]])  # dim=24

def update(self, state, action, reward, next_state, done):
    phi = self._expand_features(state)
    q_sa = self.weights[action] @ phi
    if not done:
        next_action = self._epsilon_greedy_action(next_state)
        self._cached_next_action = next_action   # <-- cache A'
        q_next = self.weights[next_action] @ self._expand_features(next_state)
```

SARSA: Implementation

Function approximation: $Q(s, a; \mathbf{w}) = \mathbf{w}_a^\top \phi(s)$

Feature vector with polynomial and interaction terms:

$$\phi(s) = \Big[\underbrace{a_t, a_y, a_z, w_t, w_y, w_z}_{8\text{ raw}}, \underbrace{t^2, w_t^2, tw_y}_{3\text{ poly}}, \underbrace{t \cdot \mathrm{onehot}(y), t \cdot \mathrm{onehot}(z)}_{6\text{ time-regime crosses}}, \underbrace{w \cdot \mathrm{onehot}(y), w \cdot \mathrm{onehot}(z)}_{6\text{ wealth-regime crosses}}, \underbrace{1}_{\text{bias}}\Big] \in$$

Separate $\mathbf{w}_a \in \mathbb{R}^{24}$ for each of 210 actions. Total parameters: $210 \times 24 = 5{,}040$.

The SARSA invariant – A' in update must equal A' executed:

```
def _expand_features(self, state):
    t, w = state[0], state[1]; cat = state[2:]   # 6 one-hots
    poly = [t**2, w**2, tw]
    return np.concatenate([state, poly, t*cat, w*cat, [1.0]])  # dim=24

def update(self, state, action, reward, next_state, done):
    phi = self._expand_features(state)
    q_sa = self.weights[action] @ phi
    if not done:
        next_action = self._epsilon_greedy_action(next_state)
        self._cached_next_action = next_action   # ***cache A'
        q_next = self.weights[next_action] @ self._expand_features(next_state)
```

[Patrick speaks – 1.5 min]

"Here's the implementation. The feature vector phi has 24 dimensions: 8 raw state values, 3 polynomial terms, 6 time-cross terms (t times each income and regime one-hot), 6 wealth-cross terms, and a bias. These interaction terms are crucial – they let the linear model learn different policies for different regimes and income states. For example, the t times z-bear cross term lets the model weight time differently when in a bear market. Without these crosses, the linear model can't represent state-dependent behavior."

"With 210 actions and 24 features, we have 5,040 total parameters – still tiny compared to DQN's 122K and PPO's 191K."

"Two implementation details: First, the TD-error clipping at plus or minus 10 – early in training, CRRA utility at near-zero consumption gives huge negative rewards that can blow up the weights. Clipping keeps training stable. Second, the caching mechanism for the SARSA invariant: the update picks A-prime and caches it, and the next select-action returns that cached action. Without this, you'd sample a different A-prime for the next step than you used in the TD target,

Training Convergence

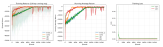
Key observations:

- **PPO** converges fastest and to the best training return (actor-critic advantage)
- **Linear Q & SARSA** converge similarly; SARSA slightly slower (on-policy)
- **DQN** slower to start (filling replay buffer) but catches up
- All agents stabilize well before training ends

Training times:

Linear Q: ∼18s, SARSA: ∼15s
DQN: ∼23 min, PPO: ∼2 min

Training Convergence



Key observations:

- **PPO** converges fastest and to the best training return (actor-critic advantage)
- **Linear Q & SARSA** converge similarly; SARSA slightly slower (on-policy)
- **DQN** slower to start (filling replay buffer) but catches up
- All agents stabilize well before training ends

Training times:
Linear Q: ~18s, SARSA: ~15s
DQN: ~23 min, PPO: ~2 min

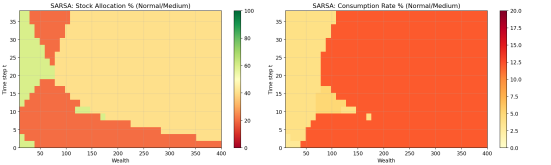
[Jeff speaks – 1 min]

"Here are the learning curves for all four agents. The y-axis is the 100-episode running average of episode returns. All four agents start with very negative returns because epsilon equals 1 means fully random actions – many of which hit the zero-consumption penalty."

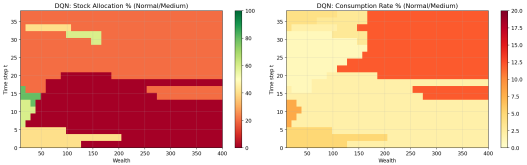
"The two linear agents (Linear Q and SARSA) converge quickly and similarly. SARSA is slightly slower because on-policy updates can only learn from the current behavior policy, not from the best possible greedy action. DQN is slowest to start because it needs to fill a 500K replay buffer before learning kicks in, but eventually produces the best evaluation utility. PPO is interesting – it often shows the best training return because entropy regularization helps it find high-consumption strategies, but evaluation utility is slightly lower."

"Training times: the linear agents take 15-18 seconds for 15K episodes – very fast. DQN takes about 23 minutes because of the neural network forward/backward passes plus replay sampling. PPO is about 2 minutes. For practical deployment, SARSA and Linear Q are the clear winners on compute efficiency."

Learned Policies: What Do the Agents Do?



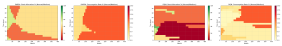
SARSA Policy



DQN Policy

Reading the heatmaps: x-axis = time (year), y-axis = wealth level. Color = stock allocation chosen by the agent. All agents learn intuitive patterns: higher stock allocation when young (time to recover from crashes), shift toward bonds/cash near retirement.

Learned Policies: What Do the Agents Do?



Reading the heatmaps: x-axis = time (year), y-axis = wealth level. Color = stock allocation chosen by the agent. All agents learn intuitive patterns: higher stock allocation when young (time to recover from crashes), shift toward bonds/cash near retirement.

[Jeff speaks – 1 min]

"These policy heatmaps show what each agent actually does at different combinations of time and wealth. The x-axis is the year, the y-axis is wealth, and the color represents the stock allocation the agent chooses."

"Both SARSA and DQN learn the same qualitative pattern that we saw in Phase 2's exact DP solution: invest heavily in stocks when young and gradually shift to bonds as retirement approaches. This is the classic glidepath shape, but unlike a fixed glidepath, the RL agents also condition on wealth – they invest more aggressively when wealth is low, because the marginal utility gain from risky growth is higher."

"The DQN policy is smoother because the neural network naturally interpolates, while SARSA's linear approximation produces sharper boundaries."

Monte Carlo Evaluation (2,000 Simulated Lifetimes)

Strategy	$\mathbb{E}$[Utility]	$\mathbb{E}[W_T]$	Med$[W_T]$	Std$[W_T]$	P_{10}	P_{90}
DQN	$-$**1.54**	115	107	52	58	186
Linear Q	-1.59	96	87	34	60	144
SARSA	-1.88	99	93	35	58	150
PPO	-1.95	91	81	43	45	150
Static 60/40	-2.63	487	470	166	281	747
Equal 1/3	-2.75	408	396	112	271	564
Glidepath 80$\rightarrow$20	-2.63	489	474	140	320	697

All four RL agents beat all three baselines on utility.

- RL agents have *lower* terminal wealth but *higher* utility – they consume more along the way
- **DQN** achieves the best utility (-1.54); **Linear Q** second at -1.59
- **SARSA** (-1.88) and **PPO** (-1.95) – still far better than any baseline (-2.63)
- Baselines accumulate large wealth (\$487+) but don't optimize consumption timing

Monte Carlo Evaluation (2,000 Simulated Lifetimes)

Strategy	$\mathbb{E}[\text{Utility}]$	$\mathbb{E}[W_T]$	$\text{Med}[W_T]$	$\text{Std}[W_T]$	P_{50}	P_{90}
DQN	**−1.54**	115	107	52	58	186
Linear Q	−1.59	96	87	34	60	144
SARSA	−1.88	99	93	35	58	150
PPO	−1.95	91	81	43	45	150
Static 60/40	−2.63	487	470	166	281	747
Equal 1/3	−2.75	408	396	112	271	564
Glidepath 80→20	−2.63	489	474	140	320	697

All four RL agents beat all three baselines on utility.

- RL agents have *lower* terminal wealth but *higher* utility – they consume more along the way
- **DQN** achieves the best utility (−1.54); **Linear Q** second at −1.59
- **SARSA** (−1.88) and **PPO** (−1.95) – still far better than any baseline (−2.63)
- Baselines accumulate large wealth ($487+) but don't optimize consumption timing

[Jeff speaks – 1.5 min]

"Here's the full comparison across all 7 strategies, evaluated on 2,000 Monte Carlo simulated lifetimes with no exploration. The headline: every RL agent beats every baseline on expected utility."

"DQN achieves the best utility at negative 1.54, followed by Linear Q at negative 1.59. These are the off-policy methods. The on-policy methods, SARSA and PPO, are slightly more conservative at negative 1.88 and negative 1.95. All baselines cluster at negative 2.63 to 2.75."

"The paradox: RL agents have much lower terminal wealth – around 90-115 versus 490 for static 60/40. But CRRA utility with gamma equals 2 rewards smooth lifetime consumption, not terminal wealth. The RL agents learned this from scratch – consume steadily, invest conservatively, and you get more utility than hoarding wealth. The baselines use a fixed 4% withdrawal rate and mostly just invest, accumulating large volatile wealth but getting less lifetime utility."

"This is the key insight: RL optimizes the actual objective. The baselines implicitly optimize wealth, which is a poor proxy for CRRA utility when the investor is risk-averse."

Risk Metrics

Strategy	MaxDD	C-Sharpe	Min W_T
DQN	60.4%	**26.53**	18.3
Linear Q	54.0%	18.30	**33.2**
SARSA	57.1%	10.25	24.9
PPO	66.1%	12.58	20.5
Static 60/40	23.8%	2.65	98.4
Equal 1/3	14.2%	2.98	126.4
Glidepath	18.1%	2.67	131.7

Consumption Sharpe ratio:

$$\text{C-Sharpe} = \frac{\text{mean}(\bar{c}_t)}{\text{std}(\bar{c}_t)}$$

Key takeaways:

All RL agents have **C-Sharpe** > 10, vs. ~ 2.7 for baselines.
$\Rightarrow$ RL consumption is 4–10$\times$ smoother.

Max drawdown is higher for RL (54–66%):

- RL agents consume more $\rightarrow$ less savings buffer
- Higher drawdown is an *accepted trade-off* for better utility

Linear Q has the highest Min W_T among RL agents ($33.2):

- Rich feature expansion $\rightarrow$ learns cautious low-wealth behavior
- SARSA ($24.9) is 2nd; on-policy avoids

Risk Metrics

Key takeaways:

Strategy	MaxDD	C-Sharpe	Min W_T
DQN	60.4%	**26.53**	18.3
Linear Q	54.0%	18.30	**33.2**
SARSA	57.1%	10.25	24.9
PPO	66.1%	12.58	20.5
Static 60/40	23.8%	2.65	98.4
Equal 1/3	14.2%	2.98	126.4
Glidepath	18.1%	2.67	131.7

- All RL agents have **C-Sharpe** > 10, vs. $\sim$2.7 for baselines.
- RL consumption is 4–10$\times$ smoother.
- **Max drawdown** is higher for RL (54–66%):
  - RL agents consume more $\rightarrow$ less savings buffer
  - Higher drawdown is an *accepted* trade-off for better utility

Consumption Sharpe ratio:

$$\text{C-Sharpe} = \frac{\text{mean}(\tilde{c}_t)}{\text{std}(\tilde{c}_t)}$$

Linear Q has the highest Min W_T among RL agents (§33.2):

- Rich feature expansion $\rightarrow$ learns cautious low-wealth behavior
- SARSA (§24.9) is 2nd: on-policy avoids

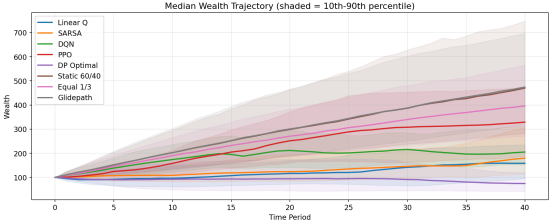
[Jeff speaks – 1 min]

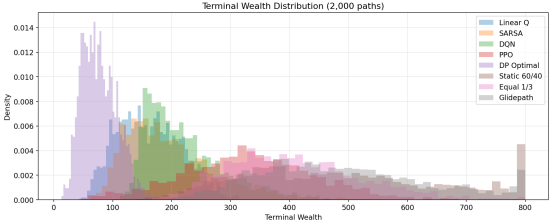
"The risk metrics reinforce the main finding. All RL agents achieve consumption Sharpe ratios above 10 – between 4 and 10 times higher than any baseline. DQN leads at 26.53. This means the RL agents deliver much smoother consumption streams over 40 years."

"The trade-off is higher max drawdown – 54 to 66 percent for RL versus 14 to 24 percent for baselines. This makes sense: more consumption leaves a smaller wealth buffer. Importantly, this is a deliberate choice the agent made – higher consumption smoothness is worth the drawdown risk under CRRA utility."

"Linear Q has the best minimum terminal wealth at 33.2 – it learned conservative behavior when wealth is low, which protects against tail risks. SARSA is second at 24.9. Interestingly, DQN has the lowest min at 18.3 despite the best utility – it takes more risk in exchange for higher expected consumption. No strategy hits ruin."

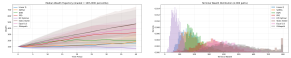
Wealth Trajectories and Terminal Wealth





Left: Sample wealth trajectories over 40 years. RL agents (consuming along the way) have lower wealth paths than baselines (which only invest). **Right:** Terminal wealth distributions. Baselines produce higher but more spread-out terminal wealth; RL agents produce tighter distributions centered on lower values – by design.

Wealth Trajectories and Terminal Wealth



Left: Sample wealth trajectories over 40 years. RL agents (consuming along the way) have lower wealth paths than baselines (which only invest). **Right:** Terminal wealth distributions. Baselines produce higher but more spread-out terminal wealth; RL agents produce tighter distributions centered on lower values – by design.

[Jeff speaks – 45 sec]

"These plots show the story visually. On the left, wealth trajectories: baselines grow to 400-500 because they never consume. RL agents stay around 100-150 because they extract consumption along the way. On the right, terminal wealth distributions: baselines have wide spreads, RL agents have tight, concentrated distributions. The RL agents deliberately traded terminal wealth for lifetime consumption smoothness."

Comparing All Four RL Agents

	Linear Q	**SARSA**	**DQN**	**PPO**
Update target	$\max_a Q(s', a)$	$Q(s', a')$	$\max_a Q(s', a)$	Policy gradient
Function approx	Linear (24-dim)	Linear (24-dim)	Dueling (256-256)	Sep. A-C (256-256)
Parameters	5,040	5,040	$\sim$122K	$\sim$191K
Training time	$\sim$18s	$\sim$15s	$\sim$23 min	$\sim$2 min
$\mathbb{E}$**[Utility]**	-1.59	-1.88	$\mathbf{-1.54}$	-1.95
C-Sharpe	18.30	10.25	**26.53**	12.58
Min W_T	**33.2**	24.9	18.3	20.5

Takeaways:

- **DQN wins on utility** (-1.54) – Dueling arch + large replay buffer captures nonlinear value
- **Linear agents win on efficiency** – 5,040 params, trains in seconds, interpretable policy
- **Off-policy** $>$ **on-policy on utility**: Linear Q beats SARSA; DQN beats PPO
- **All RL agents crush all baselines** – optimizing the right objective matters most

Comparing All Four RL Agents

	Linear Q	SARSA	DQN	PPO
Update target	$\max_a Q(s',a)$	$Q(s',a')$	$\max_a Q(s',a)$	Policy gradient
Function approx	Linear (24-dim)	Linear (24-dim)	Dueling (256-256)	Seq. A-C (256-256)
Parameters	5,040	5,040	~122K	~191K
Training time	~18s	~15s	~23 min	~2 min
Ξ[Utility]	−1.59	−1.88	**−1.54**	−1.95
C-Sharpe	18.32	10.25	**26.53**	12.58
Min W_T	**33.2**	24.9	18.3	20.5

Takeaways:

- **DQN wins on utility** (−1.54) – Dueling arch + large replay buffer captures nonlinear value
- **Linear agents win on efficiency** – 5,040 params, trains in seconds, interpretable policy
- **Off-policy** > **on-policy on utility**: Linear Q beats SARSA; DQN beats PPO
- **All RL agents crush all baselines** – optimizing the right objective matters most

[Jack speaks – 1 min]

"Here's the final comparison. The cleanest takeaway: all four RL agents beat all three baselines – optimizing the right objective matters more than which algorithm you pick."

"Among RL agents, DQN wins on utility at negative 1.54, then Linear Q at negative 1.59. Note the pattern: the off-policy methods (Linear Q, DQN) outperform the on-policy methods (SARSA, PPO). This is consistent with theory – off-policy methods can bootstrap from the greedy action, learning more aggressively. On-policy methods account for their own exploratory behavior, which makes them more conservative."

"But the linear agents have major practical advantages: 5,040 parameters versus 122K or 191K, trains in 15-18 seconds versus 23 minutes for DQN, and interpretable weights. The utility gap between the best linear agent (Linear Q, -1.59) and the best neural agent (DQN, -1.54) is tiny. For most applications, the linear agents are the pragmatic choice."

"And SARSA specifically: the on-policy update means it doesn't overestimate Q-values from exploratory actions, which is why it's slightly more conservative than Linear Q. In a financial

Conclusions

1. **Exact DP works but doesn't scale.** Phase 2's backward induction gives the provably optimal solution in 0.4s, but a 119× blowup to Phase 3 parameters makes it infeasible.
2. **RL handles continuous wealth and richer state spaces.** Function approximation replaces the DP table, scaling linearly in episodes rather than exponentially in state dimensions.
3. **All four RL agents beat all baselines on utility.** Off-policy methods (DQN, Linear Q) achieve the best utility; on-policy (SARSA, PPO) are simpler and faster. SARSA's on-policy nature gives the most conservative policy.
4. **RL optimizes the right objective.** Static strategies implicitly optimize wealth. RL directly optimizes *lifetime utility*, discovering that a risk-averse investor should consume more and invest conservatively – matching financial theory (Merton's solution).

Future work: Continuous actions (DDPG/SAC), more assets, transaction costs, real data calibration.

Conclusions

1. **Exact DP works but doesn't scale.** Phase 2's backward induction gives the provably optimal solution in 0.4s, but a 119× blowup to Phase 3 parameters makes it infeasible.
2. **RL handles continuous wealth and richer state spaces.** Function approximation replaces the DP table, scaling linearly in episodes rather than exponentially in state dimensions.
3. **All four RL agents beat all baselines on utility.** Off-policy methods (DQN, Linear Q) achieve the best utility; on-policy (SARSA, PPO) are simpler and faster. SARSA's on-policy nature gives the most conservative policy.
4. **RL optimizes the right objective.** Static strategies implicitly optimize wealth. RL directly optimizes *lifetime* utility, discovering that a risk-averse investor should consume more and invest conservatively – matching financial theory (Merton's solution).

Future work: Continuous actions (DDPG/SAC), more assets, transaction costs, real data calibration.

[Patrick speaks – 1 min]

"Let me wrap up with four takeaways. First, exact DP gives perfect answers but can't handle realistic problem sizes. Second, RL with function approximation handles the richer problem – continuous wealth, 3 regimes, 3 income states, 40-year horizon."

"Third, all four RL agents beat all three baselines. The neural network agents get the best raw utility, but the linear agents are dramatically simpler and faster. SARSA in particular produces the most conservative policy thanks to its on-policy update."

"Fourth, and most importantly: RL optimizes the actual objective. Static strategies like 60/40 produce higher wealth, but that's not what a risk-averse investor cares about. The RL agents discovered – from scratch, through trial and error – that smooth consumption beats volatile wealth. This is consistent with classical results like Merton's portfolio theory."

"For future work, we'd like to try continuous action spaces using DDPG or SAC, add more asset classes, incorporate transaction costs, and calibrate to real market data."

Thank You!

Questions?

Repository: `github.com/yuxiang-jack-zhang/CME241`

Relevant textbook sections: Ch. 4 (DP), Ch. 12.4 (SARSA), Ch. 12.6 (Q-Learning), Ch. 13 (Policy Gradient)

Thank You!

Questions?

Repository: `github.com/yuxiang-jack-zhang/OR241`
Relevant textbook sections: Ch. 4 (DP), Ch. 12.4 (SARSA), Ch. 12.6 (Q-Learning), Ch. 13 (Policy Gradient)

[All – open for questions]
Likely questions and answers:

"Why does RL have lower wealth but better utility?" – Because CRRA utility with gamma=2 is extremely risk-averse. Marginal utility of consumption diminishes fast, so smooth consumption over 40 years beats a large volatile nest egg. The math: u(c) = -1/c, so doubling consumption from 10 to 20 gives more utility gain than going from 100 to 110.

"Why is gamma=1 in the TD update if beta=0.96?" – Because beta is already baked into the rewards as beta^t * u(c_t). Using gamma=0.96 in the update would double-discount. gamma=1 with discounted rewards correctly recovers the CRRA objective.

"Why does DQN beat SARSA?" – DQN is off-policy: it uses max Q(s',a) in its target, learning from the optimal greedy policy regardless of exploration. SARSA uses Q(s', a') where a' is the exploratory action actually taken, which is more conservative. In this problem, the equity premium is strong enough that the aggressive off-policy updates pay off.

"Why not use eligibility traces (SARSA(λ))?" – With 40-step episodes, TD(0) already propagates value well. Adding traces could help early in training but adds hyperparameter complexity.